VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

WEEK-8

Simulate the following Disk scheduling algorithms. a) FCFS b) SSTF c) SCAN

AIM- To implement disk scheduling algorithms such as FCFS, SSTF, SCAN

ALGORITHM-

FCFS – First Come First Serve

- Start
- Read the number of disk requests.
- Read the request queue.
- Read the initial head position.
- Set totalMovement = 0.
- Take the requests in the given order.
- Calculate the head movement:
- $| CurrentHead - Request |$

- Add the movement to totalMovement.
- Make the current request the new head position.
- Repeat until all requests are serviced.
- Display the seek sequence and total head movement.
- Stop

2. SSTF – Shortest Seek Time First

1. **Start**
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Mark all requests as **unvisited**.
6. Find the unvisited request having the **minimum distance** from the current head.
7. Calculate:

$$Distance = | CurrentHead - Request |$$

8. Service that requests.
9. Mark the request as visited.

10. Add the distance to totalMovement.
11. Make that request the new head position.
12. Repeat until all requests are serviced.
13. Display the seek sequence and total head movement.
14. **Stop**

3. SCAN

1. **Start**
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Read the disk size.
6. Read the direction of head movement.
7. Sort the request queue in ascending order.
8. If the direction is **right**:
    - Service requests greater than the head in ascending order.
    - Move the head to the **end of the disk**.
    - Reverse the direction.
    - Service the remaining requests in descending order.
9. If the direction is **left**:
    - Service requests smaller than the head in descending order.
    - Move the head to **0**.
    - Reverse the direction.
    - Service the remaining requests in ascending order.
10. Calculate the total head movement.
11. Display the seek sequence and total head movement.
12. **Stop**

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

FCFS

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, head, i;
   int requests[50];
   int totalMovement = 0;

   printf("=========================================\n");
   printf("      DISK SCHEDULING - FCFS\n");
   printf("=========================================\n");

   printf("Enter number of requests: ");
   scanf("%d", &n);

   printf("Enter request queue:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &requests[i]);

   printf("Enter initial head position: ");
   scanf("%d", &head);

   printf("\nSeek Sequence: %d", head);
```

```
    for(i = 0; i < n; i++)
    {
        totalMovement += abs(head - requests[i]);
        head = requests[i];

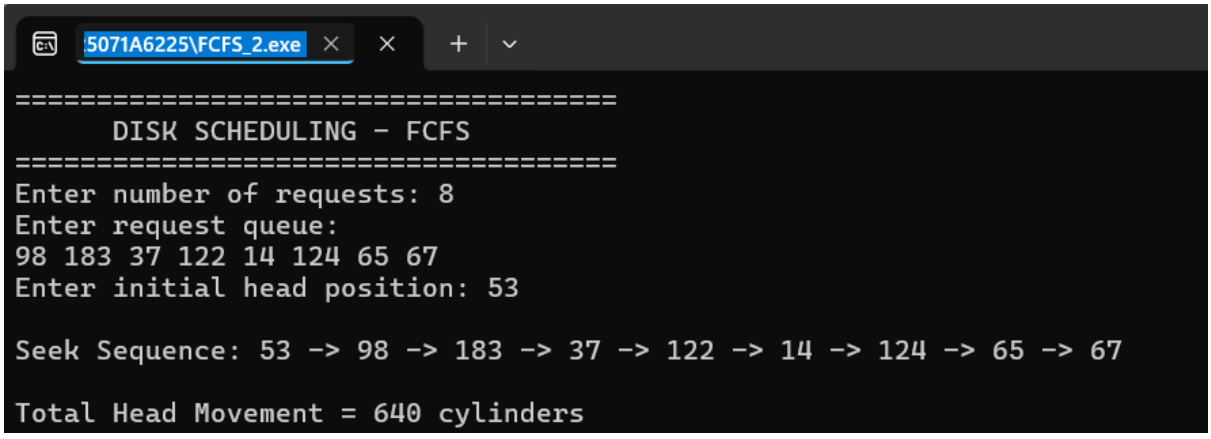
        printf(" -> %d", head);
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

    return 0;
}
```

OUTPUT:

TESTCASE 1:



TESTCASE 2:

```
5071A6225\FCFS_2.exe
====================================
      DISK SCHEDULING - FCFS
====================================
Enter number of requests: 7
Enter request queue:
 82 170 43 140 24 16 190
Enter initial head position: 50

Seek Sequence: 50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190

Total Head Movement = 642 cylinders
```

TESTCASE 3:

```
5071A6225\FCFS_2.exe
====================================
      DISK SCHEDULING - FCFS
====================================
Enter number of requests: 8
Enter request queue:
176 79 34 60 92 11 41 114
Enter initial head position: 50

Seek Sequence: 50 -> 176 -> 79 -> 34 -> 60 -> 92 -> 11 -> 41 -> 114

Total Head Movement = 510 cylinders
```

Program:

SSTF-

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, head;
   int request[50], visited[50];
   int i, count, index, distance;
   int minDistance;
   int totalMovement = 0;

   printf("=======================================\n");
   printf("       DISK SCHEDULING - SSTF\n");
   printf("=======================================\n");

   printf("Enter number of requests: ");
   scanf("%d", &n);

   printf("Enter request queue:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &request[i]);
      visited[i] = 0;
   }

   printf("Enter initial head position: ");
   scanf("%d", &head);
```

```
printf("\nSeek Sequence: %d", head);

 for(count = 0; count < n; count++)
 {
    minDistance = 99999;
    index = -1;

    /* Find nearest unvisited request */
    for(i = 0; i < n; i++)
    {
       if(visited[i] == 0)
       {
          distance = abs(head - request[i]);

          if(distance < minDistance)
          {
             minDistance = distance;
             index = i;
          }
       }
    }

    /* Seryice the nearest request */
    visited[index] = 1;
    totalMovement += minDistance;
    head = request[index];
```

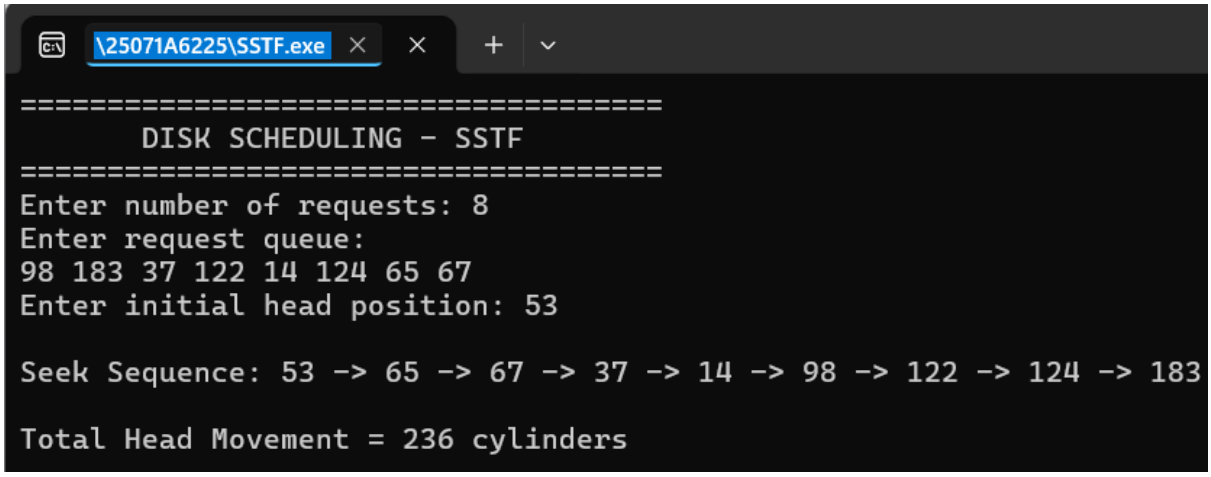
```
    printf(" -> %d", head);
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

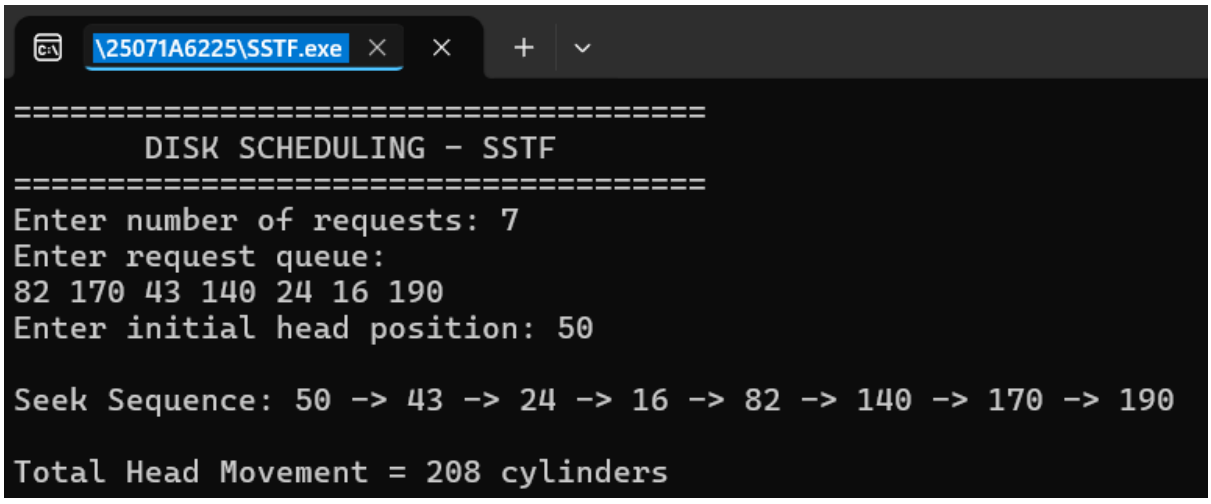
    return 0;
}
```

OUTPUT

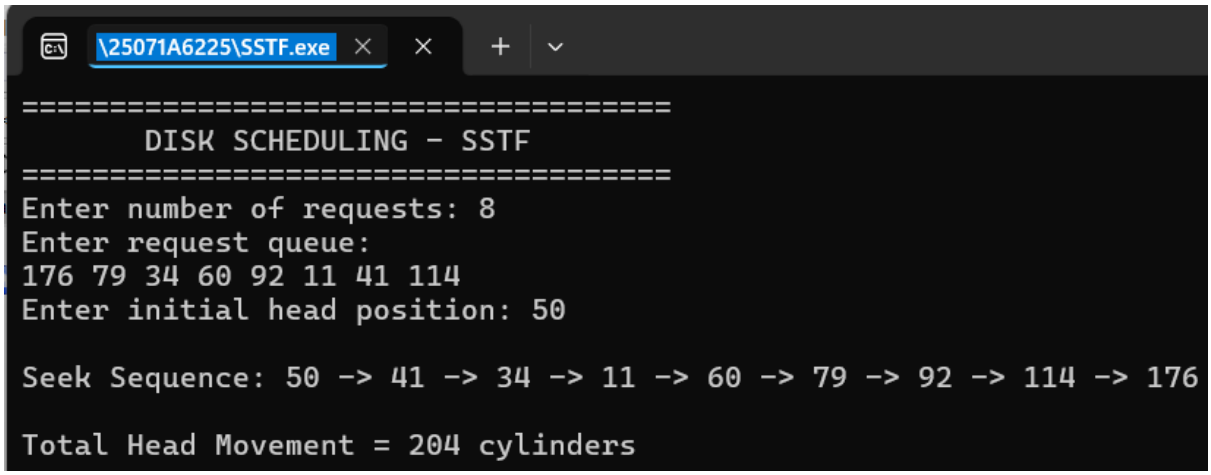
TESTCASE 1:



TESTCASE 2:



TESTCASE 3:



Program:

SCAN

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, diskSize, direction;
    int request[50];
    int i, j, temp;
    int totalMovement = 0;

    printf("=====================================\n");
    printf("       DISK SCHEDULING - SCAN\n");
    printf("=====================================\n");
```

```
printf("Enter number of requests: ");
scanf("%d", &n);

printf("Enter request queue:\n");
for(i = 0; i < n; i++)
   scanf("%d", &request[i]);

printf("Enter initial head position: ");
scanf("%d", &head);

printf("Enter disk size: ");
scanf("%d", &diskSize);

printf("Enter direction (1 = Right, 0 = Left): ");
scanf("%d", &direction);

/* Sort requests in ascending order */
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if(request[i] > request[j])
      {
         temp = request[i];
         request[i] = request[j];
         request[j] = temp;
      }
   }
}
```

```
printf("\nSeek Sequence: %d", head);

  if(direction == 1)
  {
     /* Move towards right */
     for(i = 0; i < n; i++)
     {
        if(request[i] >= head)
        {
           totalMovement += abs(head - request[i]);
           head = request[i];

           printf(" -> %d", head);
        }
     }

     /* Move to end of disk */
     if(head != diskSize - 1)
     {
        totalMovement += abs(head - (diskSize - 1));
        head = diskSize - 1;

        printf(" -> %d", head);
     }

     /* Reverse direction */
```

```
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] < head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }
}
else
{
    /* Move towards left */
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] <= head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }

    /* Move to beginning of disk */
    if(head != 0)
    {
        totalMovement += head;
```

```
    head = 0;

        printf(" -> %d", head);
    }

    /* Reverse direction */
    for(i = 0; i < n; i++)
    {
        if(request[i] > head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

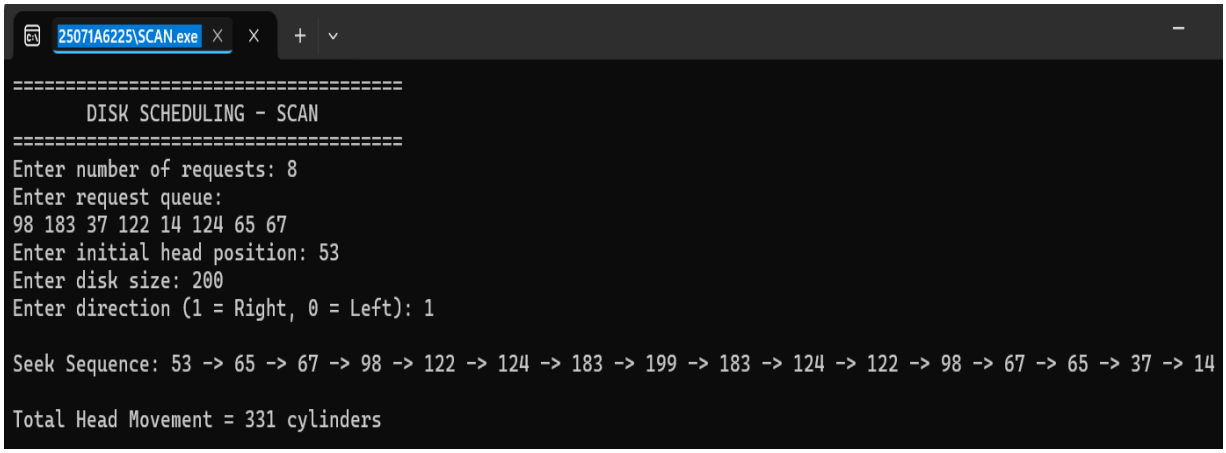
            printf(" -> %d", head);
        }
    }
  }

  printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

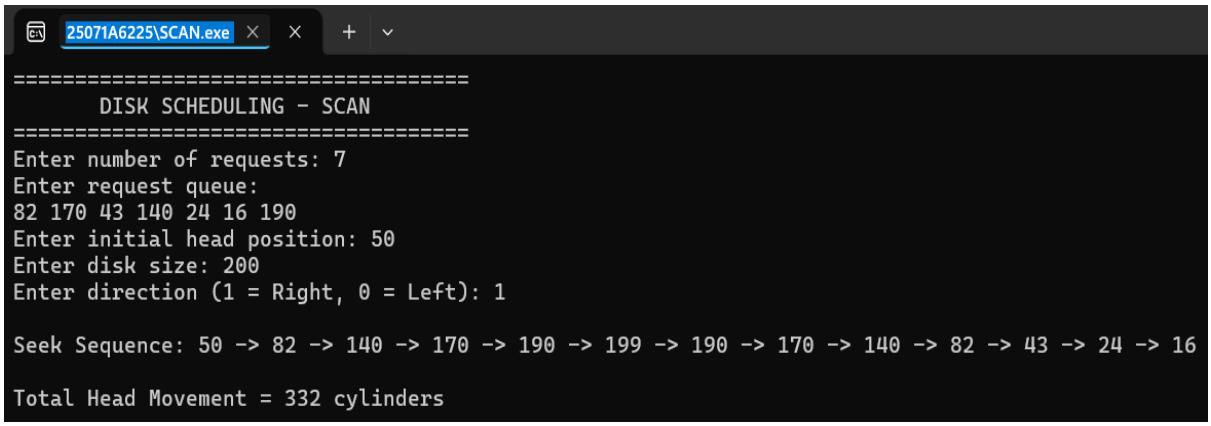
  return 0;
}
```

OUTPUT

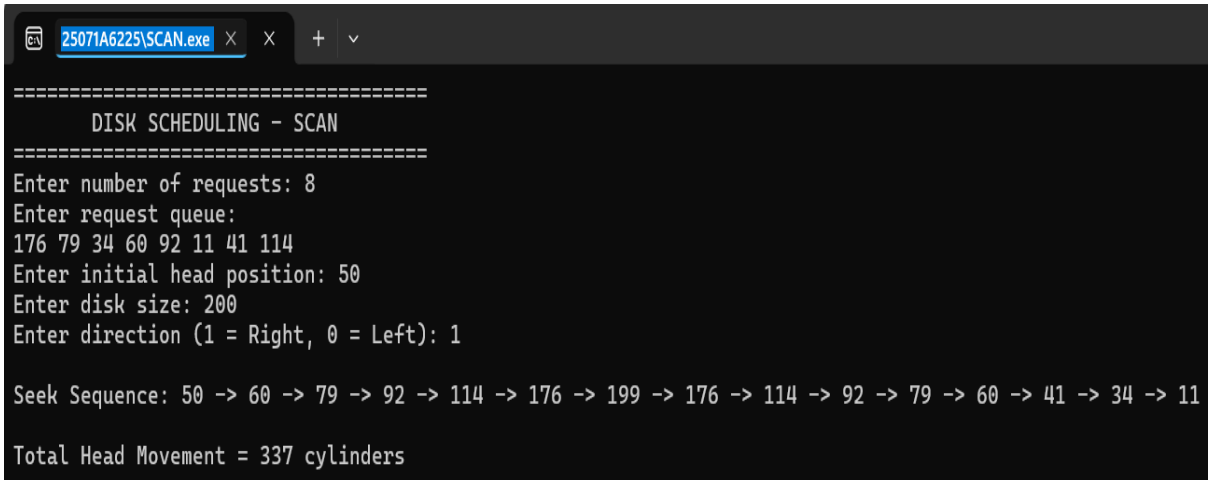
TESTCASE 1:



TESTCASE 2:



TESTCASE 3:



VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

WEEK-9

Simulate the following Disk scheduling algorithms a) C-SCAN b) LOOK c) C-LOOK

AIM- To implement disk scheduling algorithms such as C-SCAN, LOOK , C-LOOK

ALGORITHM-

C-SCAN – Circular SCAN

1. **Start**
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Read the disk size.
6. Read the direction of head movement.
7. Sort the request queue in ascending order.
8. If the direction is **right**:
    - Service requests greater than the head in ascending order.
    - Move to the **end of the disk**.
    - Move/jump to **0**.
    - Continue servicing the remaining requests in ascending order.
9. If the direction is **left**:
    - Service requests smaller than the head in descending order.
    - Move to **0**.
    - Move/jump to the **end of the disk**.
    - Continue servicing the remaining requests in descending order.
10. Calculate the total head movement.
11. Display the seek sequence and total head movement.
12. **Stop**

LOOK

1. **Start**
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Read the direction of head movement.
6. Sort the request queue in ascending order.
7. If the direction is **right**:
    - Service requests greater than the head in ascending order.
    - Stop at the **last request**.
    - Reverse direction.
    - Service remaining requests in descending order.
8. If the direction is **left**:
    - Service requests smaller than the head in descending order.
    - Stop at the **first request**.
    - Reverse direction.
    - Service remaining requests in ascending order.
9. Calculate total head movement.
10. Display the seek sequence and total head movement.
11. **Stop**

C-LOOK – Circular LOOK

1. **Start**
2. Read the number of disk requests.
3. Read the request queue.
4. Read the initial head position.
5. Read the direction of head movement.
6. Sort the request queue in ascending order.
7. If the direction is **right**:
    - Service requests greater than the head in ascending order.

- Stop at the **last request**.
- Jump to the **first request**.
- Continue servicing requests in ascending order.

8. If the direction is **left**:
   - Service requests smaller than the head in descending order.
   - Stop at the **first request**.
   - Jump to the **last request**.
   - Continue servicing requests in descending order.
9. Calculate total head movement.
10. Display the seek sequence and total head movement.
11. **Stop**

Program:

C SCAN-

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, diskSize;
    int request[50];
    int i, j, temp;
    int totalMovement = 0;

    printf("========================================\n");
    printf("      DISK SCHEDULING - C-SCAN\n");
    printf("========================================\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size: ");
    scanf("%d", &diskSize);
```

```
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if(request[i] > request[j])
      {
         temp = request[i];
         request[i] = request[j];
         request[j] = temp;
      }
   }
}

printf("\nSeek Sequence: %d", head);

/* Service requests greater than or equal to head */
for(i = 0; i < n; i++)
{
   if(request[i] >= head)
   {
      totalMovement += abs(head - request[i]);
      head = request[i];

      printf(" -> %d", head);
   }
}
```

```
    if(head != diskSize - 1)
    {
        totalMovement += abs(head - (diskSize - 1));
        head = diskSize - 1;

        printf(" -> %d", head);
    }

    /* Circular jump to beginning */
    totalMovement += head;
    head = 0;

    printf(" -> %d", head);

    /* Service remaining requests */
    for(i = 0; i < n; i++)
    {
        if(request[i] < head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);
    return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

OUTPUT

TESTCASE 1:

```
i071A6225\C_SCAN.exe
=====================================
       DISK SCHEDULING - C-SCAN
=====================================
Enter number of requests: 8
Enter request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200
Enter direction (1 = Right, 0 = Left): 1

Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37

Total Head Movement = 382 cylinders
```

TESTCASE 2:

```
i071A6225\C_SCAN.exe
=====================================
       DISK SCHEDULING - C-SCAN
=====================================
Enter number of requests: 7
Enter request queue:
82 170 43 140 24 16 190
Enter initial head position: 50
Enter disk size: 200
Enter direction (1 = Right, 0 = Left): 1

Seek Sequence: 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 0 -> 16 -> 24 -> 43

Total Head Movement = 391 cylinders
```

TESTCASE 3:

```
i071A6225\C_SCAN.exe
=====================================
       DISK SCHEDULING - C-SCAN
=====================================
Enter number of requests: 8
Enter request queue:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter disk size: 200
Enter direction (1 = Right, 0 = Left): 1

Seek Sequence: 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 199 -> 0 -> 11 -> 34 -> 41

Total Head Movement = 389 cylinders
```

Program:

LOOK:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, direction;
    int request[50];
    int i, j, temp;
    int totalMovement = 0;

    printf("=========================================\n");
    printf("       DISK SCHEDULING - LOOK\n");
    printf("=========================================\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter direction (1 = Right, 0 = Left): ");
    scanf("%d", &direction);
```

```
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if(request[i] > request[j])
      {
         temp = request[i];
         request[i] = request[j];
         request[j] = temp;
      }
   }
}

printf("\nSeek Sequence: %d", head);

if(direction == 1)
{
   /* Move right */
   for(i = 0; i < n; i++)
   {
      if(request[i] >= head)
      {
         totalMovement += abs(head - request[i]);
         head = request[i];

         printf(" -> %d", head);
      }
```

```
    }

    /* Reverse direction */
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] < head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }
}
else
{
    /* Move left */
    for(i = n - 1; i >= 0; i--)
    {
        if(request[i] <= head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }

    /* Reverse direction */
```

```
for(i = 0; i < n; i++)
        {
            if(request[i] > head)
            {
                totalMovement += abs(head - request[i]);
                head = request[i];

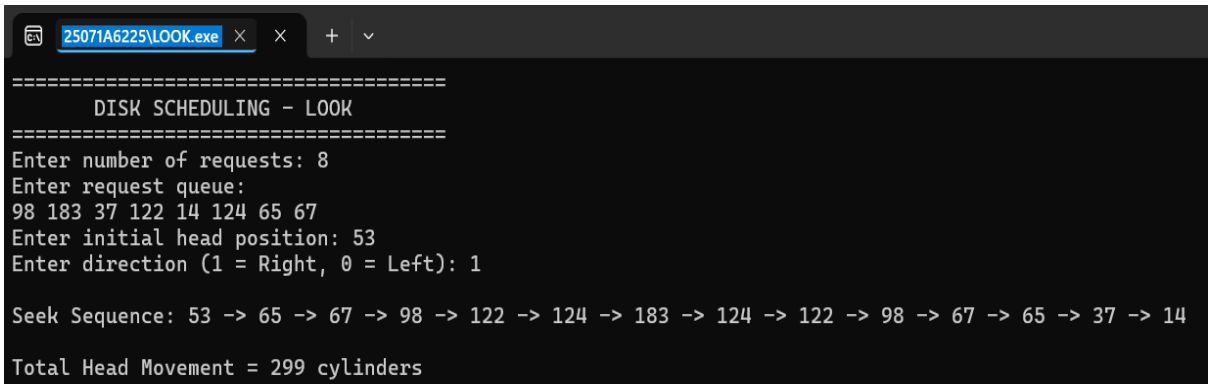
                printf(" -> %d", head);
            }
        }
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

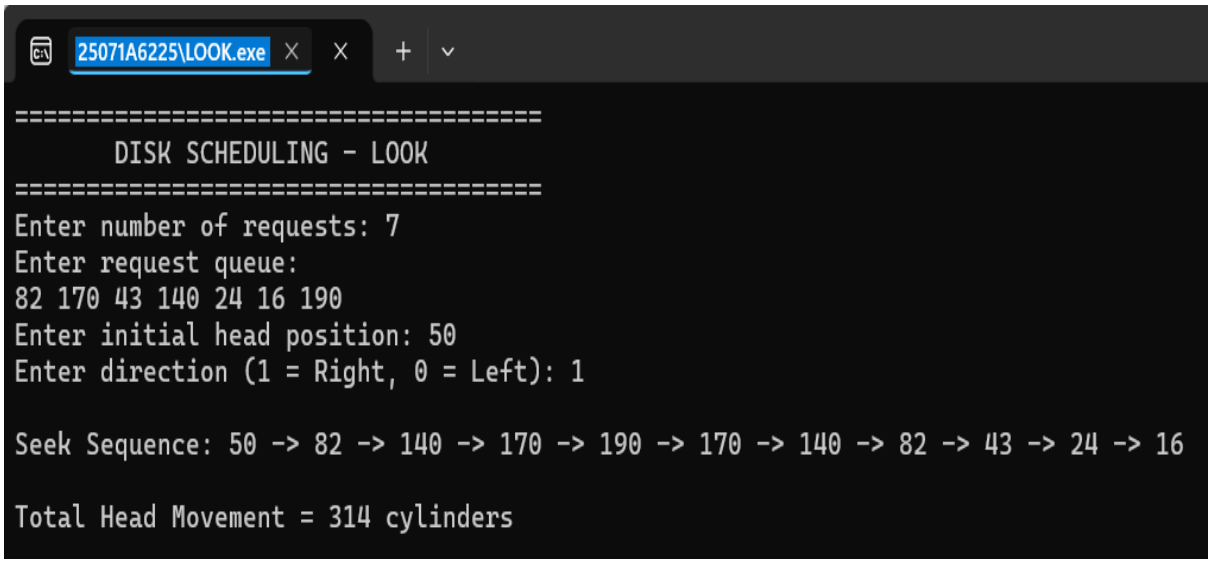
    return 0;
}
```

OUTPUT:

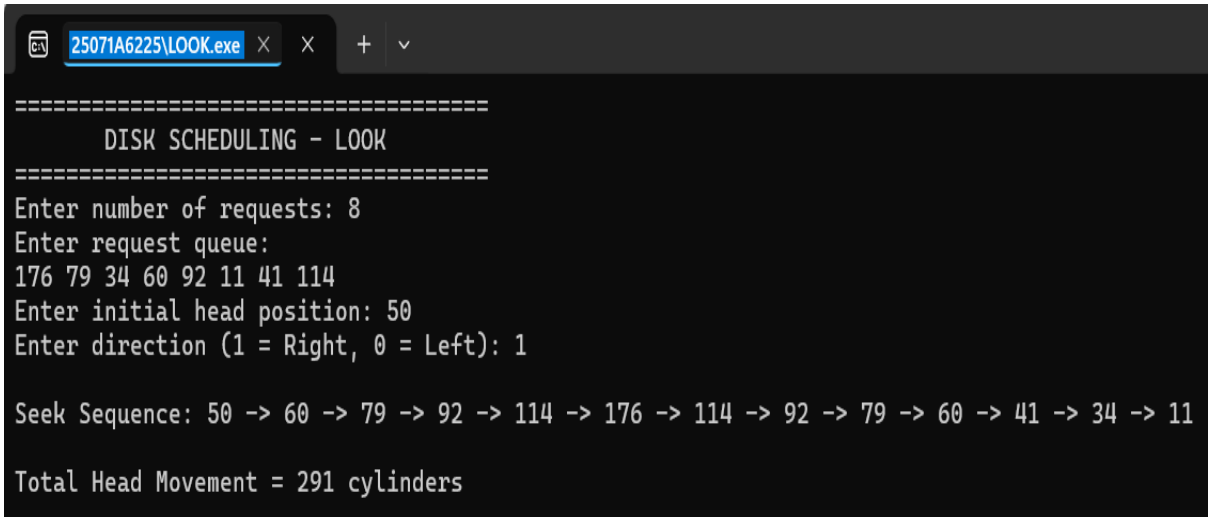
TESTCASE 1:



TESTCASE 2:



TESTCASE 3:



VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

PROGRAM:

C-LOOK:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, direction;
    int request[50];
    int i, j, temp;
    int totalMovement = 0;

    printf("========================================\n");
    printf("      DISK SCHEDULING - C-LOOK\n");
    printf("========================================\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);
```

```
printf("Enter direction (1 = Right, 0 = Left): ");
scanf("%d", &direction);

/* Sort requests */
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if(request[i] > request[j])
      {
         temp = request[i];
         request[i] = request[j];
         request[j] = temp;
      }
   }
}

printf("\nSeek Sequence: %d", head);

if(direction == 1)
{
   /* Move right */
   for(i = 0; i < n; i++)
   {
      if(request[i] >= head)
      {
         totalMovement += abs(head - request[i]);
         head = request[i];
```

```
            printf(" -> %d", head);
            }
        }
        for(i = 0; i < n; i++)
        {
            if(request[i] < head)
            {
                totalMovement += abs(head - request[i]);
                head = request[i];

                printf(" -> %d", head);
            }
        }
    }
    else
    {
        for(i = n - 1; i >= 0; i--)
        {
            if(request[i] <= head)
            {
                totalMovement += abs(head - request[i]);
                head = request[i];

                printf(" -> %d", head);
            }
        }
```

```
        for(i = n - 1; i >= 0; i--)
        {
            if(request[i] > head)
            {
                totalMovement += abs(head - request[i]);
                head = request[i];

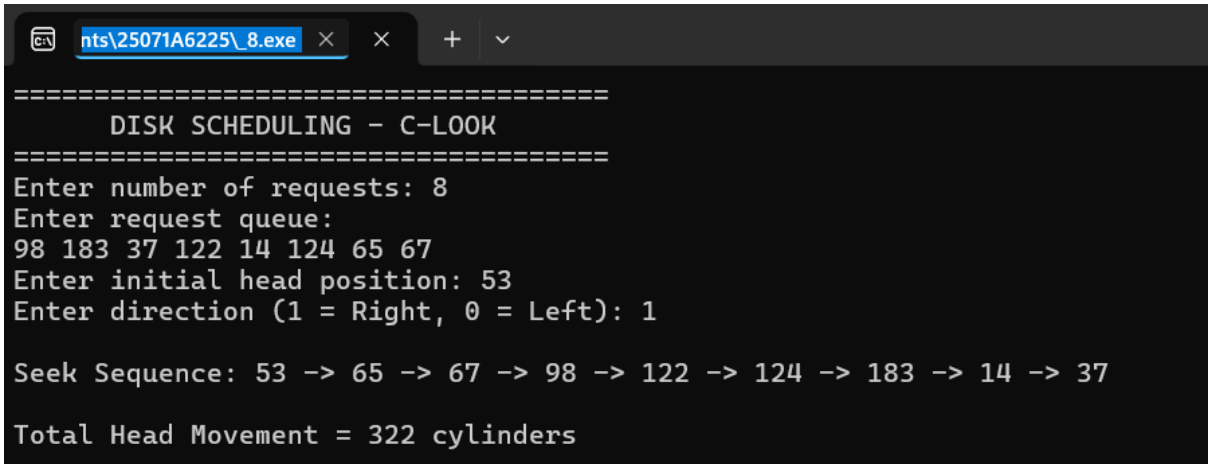
                printf(" -> %d", head);
            }
        }
    }

    printf("\n\nTotal Head Movement = %d cylinders\n", totalMovement);

    return 0;
}
```

OUTPUT:

TESTCASE 1:



TESTCASE 2:

```
nts\25071A6225\_8.exe
=====================================
      DISK SCHEDULING - C-LOOK
=====================================
Enter number of requests: 7
Enter request queue:
82 170 43 140 24 16 190
Enter initial head position: 50
Enter direction (1 = Right, 0 = Left): 1

Seek Sequence: 50 -> 82 -> 140 -> 170 -> 190 -> 16 -> 24 -> 43

Total Head Movement = 341 cylinders
```

TESTCASE 3:

```
nts\25071A6225\_8.exe
=====================================
      DISK SCHEDULING - C-LOOK
=====================================
Enter number of requests: 8
Enter request queue:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter direction (1 = Right, 0 = Left): 1

Seek Sequence: 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 11 -> 34 -> 41

Total Head Movement = 321 cylinders
```